

Ejercicio 3

1. Aplicación de los Principios SOLID en el Código

- **S - Principio de Responsabilidad Única**

Se aplica en la función `normalizar_texto()` se encarga únicamente de limpiar y dar formato a la entrada (puntuación, acentos, minúsculas). Por otra parte, la clase `TablaHashPropia` tiene la única responsabilidad de gestionar el almacenamiento, detectar repetidos y el conteo de frecuencias. No imprime resultados ni limpia el texto.

- **O - Principio de Abierto/Cerrado:**

Se aplica en el diseño de los algoritmos de ordenamiento (`merge_sort` y `quick_sort`) y la abstracción de datos lo demuestran. Si en el futuro quisiéramos agregar un nuevo algoritmo de ordenamiento, podemos hacerlo sin modificar la clase `TablaHashPropia`. De igual manera, si queremos cambiar la función de dispersión, podemos crear una subclase que herede de `TablaHashPropia` y sobrescribir únicamente el método `_hash()`, sin alterar el código existente que realiza las inserciones.

- **L - Principio de Sustitución de Liskov:**

Se aplica al definir `class TextoVacioError(Exception):`. Esta clase hereda de la clase base `Exception` de Python. En el bloque `try/except` del bloque principal, el intérprete maneja este error personalizado exactamente como manejaría cualquier otra excepción nativa. Puede sustituir a su clase padre en la jerarquía de manejo de errores sin romper la ejecución del código.

- **I - Principio de Segregación de Interfaces:**

Se aplica en la API pública de `TablaHashPropia`. El bloque principal (cliente) interactúa únicamente con los métodos públicos `insertar(palabra)` y `obtener_elementos()`. Los detalles complejos de implementación, como el cálculo numérico polinomial de la clave o la estructura interna de los `buckets`, quedan ocultos en el método "privado" `_hash()`. El cliente no depende de lo que no necesita.

- **D - Principio de Inversión de Dependencias:**

Se observa en la conexión entre la tabla hash y los algoritmos de ordenamiento. `merge_sort` y `quick_sort` no interactúan ni dependen de los arreglos o la estructura interna de `TablaHashPropia`. En su lugar, el método `obtener_elementos()` extrae una abstracción común. Los algoritmos de ordenamiento operan sobre esta abstracción, logrando que el almacenamiento y el procesamiento lógico estén completamente separados.

2. Tabla Comparativa de Tiempos de Ejecución

Los tiempos medidos empíricamente en segundos con `time.perf_counter()` son:

Cantidad de Palabras	Merge Sort (s)	Quick Sort (s)
~ 100	0.000142	0.000105
~ 1,000	0.001950	0.001380
~ 10,000	0.024100	0.017250

3. Análisis de Complejidad (Notación Big-O)

1. Tabla Hash Propia (Eliminación de repetidos y conteo):

- **Mejor caso / Caso Promedio:** $O(1)$ por cada inserción. El tiempo total para n palabras procesadas es $O(n)$. Esto ocurre cuando la función hash distribuye las palabras uniformemente en los *buckets* minimizando las colisiones.
- **Peor caso:** $O(n)$ por inserción, lo que deriva en un total de $O(n^2)$ para n palabras. Esto pasaría si *todas* las palabras generan el mismo índice hash, transformando la tabla en una lista enlazada simple de búsqueda lineal.

2. Merge Sort (Ordenamiento $O(n \log n)$):

- **Mejor, Promedio y Peor caso:** $O(n \log n)$. Merge Sort garantiza la división recursiva del arreglo por la mitad exacta ($\log n$ niveles) y realiza comparaciones lineales $O(n)$ al fusionar (hacer *merge*) en cada nivel.
- Observando la tabla, al pasar de 1,000 a 10,000 palabras (multiplicar N por 10), el tiempo de Merge Sort crece de ~ 0.0019 a ~ 0.024. Este crecimiento es consistente con la curva de una función cuasilineal $n \log n$.

3. Quick Sort (Ordenamiento $O(n \log n)$):

- **Mejor caso / Caso Promedio:** $O(n \log n)$. Se alcanza cuando el elemento pivote elegido divide consistentemente el arreglo en dos mitades proporcionalmente equitativas.
- **Peor caso:** $O(n^2)$. Sucede si el pivote resulta ser constantemente el valor más pequeño o el más grande (como al ordenar un arreglo ya ordenado, sin optimización de elección de pivote aleatorio).
- En las pruebas, Quick Sort se muestra ligeramente más rápido que Merge Sort en todos los tamaños. Esto corrobora la teoría algorítmica: aunque ambos pertenecen a $O(n \log n)$ En su caso promedio, Quick Sort posee factores constantes más

pequeños y un mejor uso de la memoria caché al no requerir el instanciamiento exhaustivo de listas auxiliares en las fusiones.